DuckDB from the Command Line

A Second Way to Talk to Your Database

Table of Contents

Why Learn the Command Line?

So far, you've used DuckDB **inside Marimo notebooks**. That's great for building and sharing analysis.

The **command line** is different — it's a fast, no-frills way to:

- Quickly check a table without opening a notebook
- Run a `.sql` file as part of a script or pipeline
- Peek inside a `.csv` file in seconds
- Practice SQL the way many working analysts actually do

👉 Notebooks and the command line are **two tools for the same job**. Knowing both makes you more flexible.

Two Different "DuckDBs"

You already have the **DuckDB Python package** installed (`pip install duckdb`). That's what powers `import duckdb` in Marimo.

Today we install a **second, separate thing**: the **DuckDB CLI** — a standalone program called `duckdb` that you run directly in your terminal, with no Python involved at all.

	Python package	DuckDB CLI
How you use it	`import duckdb` in a script/notebook	Type `duckdb` in a terminal
Needs Python?	Yes	No
Best for	Notebooks, analysis with pandas	Quick checks, scripts, learning SQL

What You'll Need

- A terminal:
  - **Mac** → Terminal app
  - **Windows** → PowerShell or Command Prompt
- 5 minutes to install the DuckDB CLI (next slides)
- The sample file `data/students.csv` in this folder

That's it — no server, no account, no configuration file.

Installing the DuckDB CLI — macOS

If you have Homebrew installed:

```
brew install duckdb
```

Don't have Homebrew? Install it first from **brew.sh**, then run the command above. Homebrew keeps `duckdb` up to date for you.

Installing the DuckDB CLI — Windows

Easiest way, using `winget` (built into Windows 10/11):

```
winget install DuckDB.cli
```

If `winget` isn't available:

1. Go to **duckdb.org/docs/installation**
2. Download the Windows CLI `.zip`
3. Unzip it — you'll get a file named `duckdb.exe`
4. Move it somewhere permanent and add that folder to your **PATH**
   (search "Edit environment variables" in Windows)

Verify the Install

Close and reopen your terminal, then run:

```
duckdb --version
```

You should see something like:

```
v1.1.3 19864453f7
```

👉 If you see a version number, you're ready. If you see `command not found`, your terminal doesn't know where `duckdb` is yet — double-check the install step above.

Starting DuckDB — In-Memory

Just type `duckdb` with nothing after it:

```
duckdb
```

```
v1.1.3 19864453f7
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
D
```

The `D` is your **prompt** — DuckDB is waiting for a command.

⚠️ "In-memory" means: when you close this window, **everything you created is gone**. Good for quick experiments.

Starting DuckDB — Persistent File

To **save** your work to disk, give `duckdb` a filename:

```
duckdb my_class.db
```

- If `my_class.db` doesn't exist yet, DuckDB **creates** it.
- If it already exists, DuckDB **reopens** it — your tables are still there.

👉 This is exactly like the difference between `duckdb.connect(':memory:')` and `duckdb.connect('my_class.db')` in your Marimo notebooks.

The DuckDB Prompt

Once you see the `D` prompt, you can type SQL directly:

```
D SELECT 6 * 7 AS answer;
┌─────────┐
│ answer  │
│ int32   │
├─────────┤
│      42 │
└─────────┘
```

Press **Enter** after the `;` to run the statement.

Don't Forget the Semicolon

If you forget the `;`, DuckDB just waits for more input:

```
D SELECT 6 * 7 AS answer
   ...>
```

Type the `;` on the next line and press Enter to finish it.

⚠️ This is the #1 "why isn't anything happening?" moment for beginners. When the prompt changes to `...>`, DuckDB is still listening — it just needs your semicolon.

Dot-Commands — Your Toolbox

Commands that start with a **dot (`.`)** are not SQL — they're DuckDB CLI shortcuts. No semicolon needed.

```
.help       show all dot-commands
.tables     list tables in the current database
.schema     show CREATE TABLE statements
.mode       change how results are displayed
.headers    show/hide column names
.timer      show how long each query took
.quit       exit the CLI
```

👉 Think of dot-commands as **settings for your session**, and SQL as **questions for your data**.

.tables and .schema

Create a table, add some rows, then inspect it:

```
D CREATE TABLE students (id INTEGER, name VARCHAR, gpa DOUBLE);
D INSERT INTO students VALUES
    (1, 'Alice', 3.8), (2, 'Bob', 3.5), (3, 'Charlie', 3.9),
    (4, 'Diana', 3.2), (5, 'Ethan', 2.9), (6, 'Fiona', 3.7),
    (7, 'George', 3.1), (8, 'Hana', 3.95);
D .tables
students
D .schema students
CREATE TABLE students(id INTEGER, name VARCHAR, gpa DOUBLE);
```

`.tables` answers *"what tables exist?"*

`.schema` answers *"what do they look like?"*

👉 The next few slides keep querying this same `students` table — it stays alive for the rest of your session.

.mode — Changing How Results Look

The default look (`box`, shown so far) is nice to read, but not easy to paste elsewhere. Switch styles with `.mode`:

```
D .mode csv
D SELECT * FROM students LIMIT 2;
id,name,gpa
1,Alice,3.8
2,Bob,3.5
```

Other useful modes: `.mode markdown` (great for pasting into a README), `.mode json`, `.mode line`. Switch back anytime with `.mode box`.

.headers and .timer

```
D .headers off
D SELECT name FROM students LIMIT 1;
Alice
```

```
D .timer on
D SELECT COUNT(*) FROM students;
┌──────────────┐
│ count_star() │
├──────────────┤
│            8 │
└──────────────┘
Run Time (s): real 0.001 user 0.000539 sys 0.000315
```

`.timer on` is handy once your tables get big and you want to know if a query is fast or slow.

Querying a CSV — No Loading Step

This is DuckDB's superpower, and it works the same on the command line as it does in a notebook. No `CREATE TABLE`, no import wizard — the file **is** the table:

```
D SELECT major, ROUND(AVG(gpa), 2) AS avg_gpa
  FROM read_csv('data/students.csv')
  GROUP BY major
  ORDER BY avg_gpa DESC;
```

```
┌──────────────────────┬───────────┐
│        major         │ avg_gpa │
├──────────────────────┼───────────┤
│ Information Systems  │    3.83 │
│ Accounting           │    3.50 │
│ Marketing            │    3.35 │
│ Finance              │    3.35 │
└──────────────────────┴───────────┘
```

One-Off Queries with `-c`

Sometimes you don't want to open the interactive prompt at all — you just want **one quick answer**. Run it straight from your terminal (not from inside `duckdb`) using `-c`:

```
duckdb -c "SELECT COUNT(*) FROM read_csv('data/students.csv')"
```

```
┌──────────────┐
│ count_star() │
├──────────────┤
│            8 │
└──────────────┘
```

👉 Great for quick checks without leaving your normal terminal workflow.

Running a Script File

1. Create the file. Open any plain-text editor (VS Code, Notepad, TextEdit — not Word), type the SQL below, and save it as `report.sql` in this same folder:

```
SELECT major, COUNT(*) AS n
FROM read_csv('data/students.csv')
GROUP BY major;
```

2. Run it. Back in your terminal (not inside the `duckdb` prompt):

```
duckdb -c ".read report.sql"
```

👉 `report.sql` is just text — DuckDB reads it and runs each statement in order. Save once, rerun anytime: that's **repeatable** SQL.

Leaving DuckDB

Any of these will exit the interactive prompt:

```
D .quit
```

```
D .exit
```

Or press **Ctrl+D** (Mac/Linux) or **Ctrl+Z then Enter** (Windows).

⚠️ If you used an **in-memory** session, everything you built disappears the moment you exit. Use a file (`duckdb my_class.db`) if you want it to survive.

Common Mistakes

- **Forgetting the `;`** → prompt hangs at `...>`. Type `;` + Enter.
- **Typing a dot-command with a `;`** → dot-commands don't need one (`.tables;` will error).
- **Quoting strings incorrectly** → SQL text needs single quotes: `WHERE major = 'Finance'`, not double quotes.
- **Using an in-memory session, then closing the terminal** → your table is gone. Use a `.db` file if you need it saved.
- **Running `duckdb` from the wrong folder** → `read_csv('data/students.csv')` only works if your terminal is in this tutorial's folder. Check with `pwd` (Mac) or `cd` (Windows).

Cheat Sheet

Command	What it does
`duckdb`	start in-memory session
`duckdb file.db`	start/reopen a persistent database
`duckdb -c "SQL"`	run one query from your terminal, then exit
`duckdb -c ".read file.sql"`	run a whole script file
`.tables`	list tables
`.schema [table]`	show table structure
`.mode box | csv | markdown | json`	change output style
`.headers on | off`	show/hide column names
`.timer on | off`	show query run time
`.quit` / `.exit`	leave DuckDB

Practice Exercise

In your terminal, `cd` into this tutorial's folder, then:

1. Start DuckDB: `duckdb`
2. Run: `SELECT * FROM read_csv('data/students.csv');`
3. Find the average GPA **per major**, highest first
4. Switch to `.mode markdown` and rerun your query
5. Save your query in a file called `my_query.sql`,
   then run it with `duckdb -c ".read my_query.sql"`
6. Exit with `.quit`

👉 If you can do all six steps, you can use DuckDB from the command line with confidence.

You're Ready

Notebooks for building analysis.

The command line for quick answers and repeatable scripts.

Resources

duckdb.org/docs/api/cli — Official CLI documentation

duckdb.org/docs/installation — Install DuckDB for any OS

OMIS 105 — Introduction to Database Management Systems — Fall 2026